Predicting vessel arrivals and planning berths for Just In Time port arrival

An AIS based ETA model integrated with a berth planning API aligned with the DCSA Port Call standard

Rohit Varadaraj
Independent project, Bengaluru, India • rohitvraj11@gmail.com
Code and documentation: https://github.com/Rohit-v-raj/ship-eta-berth-scheduler

Abstract. Ships often sail at full speed only to wait at anchor for a free berth. IMO estimates that ships spend up to 9% of their time waiting at anchorage, and an IMO study estimated that Just In Time arrival could cut container ship fuel use and CO_2 emissions by about 14% per voyage [1, 2]. Just In Time arrival needs a port to know when each ship will arrive, and to turn that knowledge into a berth plan the ship can act on. This note describes a working prototype of both. First, from about 30 million AIS position reports (Norwegian west coast, July to October 2020), I build 688 labelled approach voyages without manual labelling and train a gradient boosted tree model that predicts arrival within 69 minutes on average for 36 vessels never seen in training, 57% better than a distance over speed estimate. A deep sequence model and two features recommended in the literature were tested and rejected. Second, I integrate the model with a berth planning API built as the marine side of a terminal operating system: a MySQL data model of berths, vessel visits and allocations; berth allocation with a virtual queue; requested arrival times with speed advice that tells queued ships to slow down rather than wait at anchor; and the operator's accept, reject or omit reply. Timestamps follow the four time types of IMO FAL.5/Circ.52 and are exchanged in the DCSA Port Call 2.0 event format, the industry standard for Just In Time port calls. The fuel saved by the speed advice has not yet been measured; the note closes with that and the other limitations.

1 Motivation

A common pattern in port calls is "hurry up and wait": a ship keeps its service speed, arrives before a berth is ready, and waits at anchor with engines and generators running. Because propulsion power rises roughly with the cube of speed, the same arrival time reached at a lower speed needs much less fuel. Just In Time (JIT) arrival puts this into practice: the port tells the ship when its berth will be ready, and the ship adjusts its speed to arrive then [1].

The emissions case is substantial. A 2022 study for the IMO Low Carbon Global Industry Alliance, using 2019 AIS data for container ships, estimated mean fuel savings of 14.2% when the whole voyage is optimised, and 5.9% when only the last 24 hours are [2]. The second figure matters here: even a port that can only see ships on their final approach can deliver a measurable saving, provided it can predict arrivals and plan berths reliably over that horizon.

Making this work in practice also needs a common language between ship and terminal. The Digital Container Shipping Association (DCSA) publishes a Port Call standard for exchanging berth and service times, and presents Just In Time practice as the way for vessels to optimise speed and lower fuel use and emissions [9]. IMO's guidelines for port call data set out the estimated, requested, planned and actual times that such an exchange carries [8].

This project builds and connects the pieces a port needs for that:

1. **A reproducible data pipeline** that turns raw AIS reports into labelled approach voyages, including handling of hidden missing values and spurious "arrivals".
2. **A leakage aware ETA model**, evaluated on vessels it has never seen, with every rejected alternative kept in the record.
3. **A berth planning API**, built as the marine side of a terminal operating system and integrated with the model, that converts each prediction into a berth or queue position, a requested arrival time and a speed advisory, and exchanges them with ships as DCSA Port Call 2.0 events (Section 7).

2 Data

The raw data are AIS position reports from the Norwegian west coast (roughly 61 to 63°N and 4 to 8°E), from 27 July to 21 October 2020: about 30 million rows in 40 daily Parquet files. Each report carries the vessel identifiers (MMSI, IMO number), length, timestamp, position, speed over ground (SOG), course over ground (COG), heading and navigational status. Reports arrive about every 3 seconds per vessel.

The files contain no empty cells, but AIS encodes "not available" as plausible looking numbers (Table 1). Left in place, a speed of 102.3 knots would be modelled as real. Every row passes through one cleaning function that converts these values to missing before any other step.

Table 1. AIS sentinel values found in the data and converted to missing values.

Field	Sentinel value	Meaning
Speed over ground	102.3	speed not available
Course over ground	360.0	course not available
True heading	511	heading not available
Vessel length	0	length unknown

3 Constructing the prediction problem

There is no arrival label in AIS, so the target has to be derived. Figure 1 shows the full path from raw reports to a berth plan.

Arrival events. An arrival is a run of reports with navigational status 5 (moored). Moored reports separated by more than one hour start a new event; events with fewer than 10 reports are discarded as transient. The berth location is the median position of the event.

Approach voyages. For each arrival, the approach is the track from the end of the vessel's previous mooring, or at most 24 hours before arrival, up to the arrival time. Inspecting real voyages showed many coastal vessels that are "moored" where they started the day without having travelled anywhere. An approach is therefore kept only if it starts at least 3 nautical miles from the berth, the vessel exceeds 3 knots at some point, and at least 10 points remain after resampling.

Resampling and label. Each approach is resampled to one point per 60 seconds, and every point is labelled with y, the minutes remaining until arrival.

Features. Because a port knows where an arriving ship is going, the features are conditioned on the destination: great circle distance d to the berth (nautical miles), initial bearing to the berth, SOG, COG, latitude and longitude. A physics prior is added:

$$\hat{y}_{\text{naive}} = \min\left(\frac{60\,d}{\max(v,\,0.5)},\;6000\right)\ \text{minutes}, \tag{1}$$

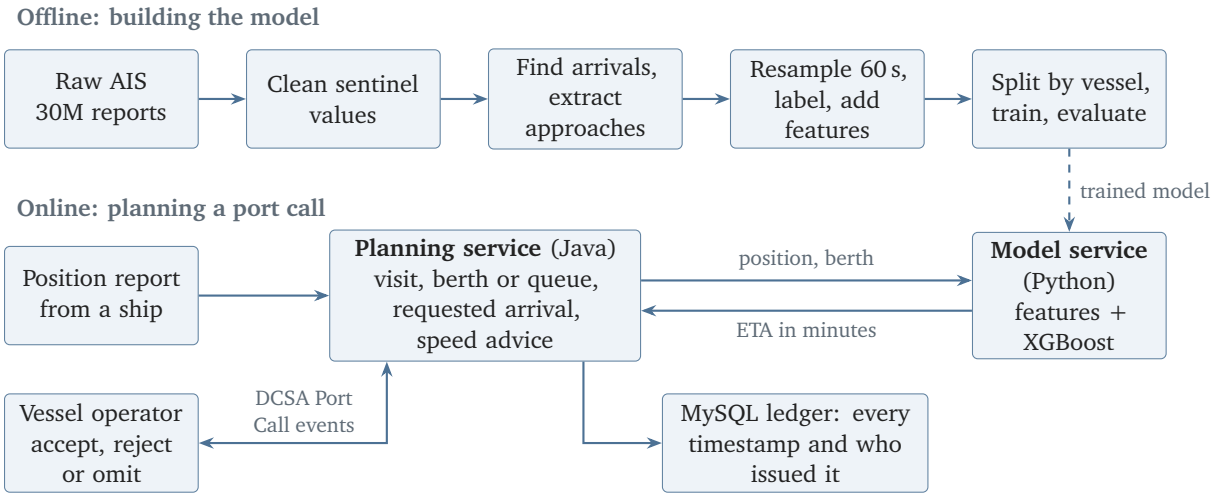


Figure 1. System overview. The offline pipeline turns raw AIS reports into a model trained and tested on separate vessels. Online, each position report is scored by the model service, and the planning service turns the ETA into a berth or queue position, a requested arrival time and a speed advisory, recording every timestamp.

where v is SOG in knots (missing values filled with the median for this feature only). Raw berth coordinates are deliberately excluded: they let a model memorise individual berths instead of learning relationships that transfer to new ones.

The result is 688 approach voyages, about 202,000 labelled points, from 253 vessels. Voyages last a few hours on average.

4 Evaluation design

Consecutive AIS reports from one voyage are nearly identical. If rows are split randomly, near copies of the same voyage land in both the training and test sets, the model is in effect tested on data it has seen, and every metric looks better than it is. One 2024 journal study reviewed for this project used such a random row split on reports a few seconds apart, which makes its reported errors hard to interpret.

Here the split is **by vessel**: each MMSI is hashed deterministically into training, validation or test (70/15/15), so every ship is wholly in one group. This gives 180 training, 37 validation and 36 test vessels (147,494 training points). The test set therefore measures what matters in operation: performance on ships the model has never seen. The validation set is used only for early stopping.

The primary metric is mean absolute error (MAE) in minutes, with RMSE and R^2 reported alongside. Error is also broken down by how far the ship is from arrival, since a planner needs to know how far ahead a prediction can be trusted.

5 Models

Physics prior. The distance over speed estimate in Equation (1), used on its own.

XGBoost [6]. A gradient boosted tree ensemble on the seven snapshot features of Section 3: squared error objective, maximum depth 6, learning rate 0.05, row and column subsampling 0.8, minimum child weight 5, L2 regularisation 1. Up to 600 trees were allowed, with early stopping after 30 rounds without improvement in validation MAE; training stopped at 305 trees. Missing speeds are handled natively by the trees. The model was chosen as the first candidate because a recent review of 29 ETA studies found tree ensembles among the strongest and most widely used

methods for this task [3].

Bidirectional GRU. Li, Jiao and Yang [4] compared twelve methods for AIS trajectory prediction and found the bidirectional GRU the most consistent performer. To test whether recent motion history improves ETA, I trained a Bi-GRU on sliding windows of 12 consecutive points (12 minutes) with a stride of 4, giving 49,018 windows (35,698 / 6,989 / 6,331 for training, validation and test, still split by vessel). Each step has nine features: angles encoded as sine and cosine pairs so that 359° and 1° are neighbours, and the remaining features standardised with training set statistics only. The network has two bidirectional layers of 64 units per direction, dropout 0.5 and a small dense head, trained with Adam (learning rate 10^{-3}, batch 256) and early stopping.

6 Results

Table 2 summarises the comparison. On all test points, XGBoost reduces MAE from 161.5 to 69 minutes (57%) and explains 62% of the variance in time to arrival. The headline gain is flattered by one weakness of the baseline: distance over speed fails on slow or stationary reports, which are about a quarter of the data. On moving reports alone, the reduction is 18% (67.1 against 82.3 minutes). Both numbers are reported because the second is the fairer test of what the model adds.

Table 2. Test results on vessels unseen in training. The Bi-GRU predicts only at window end points, so it is compared with XGBoost on exactly those points.

Model	Test points	MAE (min)	R^2
Physics prior (distance over speed)	all	161.5	
XGBoost	all	69.0	0.62
Physics prior	moving reports only	82.3	
XGBoost	moving reports only	67.1	
XGBoost	window end points	68.4	
Bi-GRU	window end points	74.9	

Error grows with the time remaining (Figure 2): about 45 minutes when the ship is under an hour out, rising to about 112 minutes beyond four hours. This is the expected shape, since more can change over a longer horizon, and it indicates the model has learned real structure rather than noise. For planning, it also means predictions made more than four hours out need wide buffers.

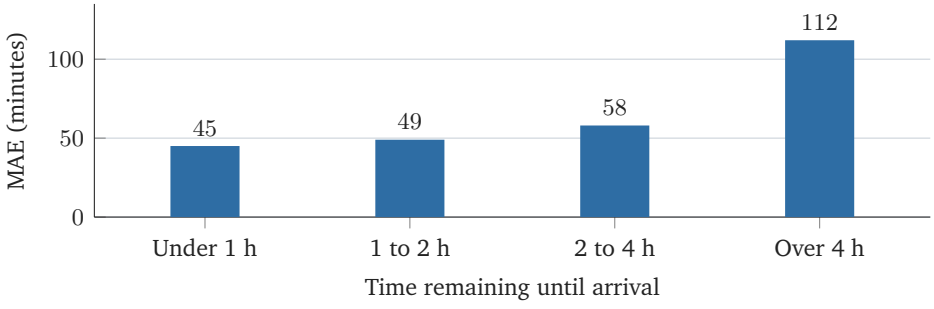


Figure 2. XGBoost test error by time remaining until arrival (values rounded to the nearest minute).

The model corrects physics rather than replacing it. By gain, the physics prior accounts for 0.51 of feature importance, distance to the berth 0.14 and bearing 0.08, with position, speed and course at 0.06 to 0.07 each. The trees learn corrections to a physical estimate, such as slowing near port and detours.

Rejected alternatives. Recommendations from the literature were treated as hypotheses and

tested against held out error (Table 3). Vessel length, a standard ETA feature [3], made MAE worse (69 to 75 minutes) on this small, mostly coastal fleet. Time of day and season indicators, following Abdi and Amrit [5], changed MAE by only about half a minute either way across seeded runs, and the data cover only summer and autumn, so the season flag carries little information. The Bi-GRU lost to XGBoost on identical points. With 688 voyages, a data hungry sequence model has little to work with, and a single snapshot that includes distance and a physics prior already captures most of the signal. The trajectory result in [4] was for a different task with far more data.

Table 3. Alternatives tested and rejected. All runs use the same vessel split.

Change	Effect on test MAE	Decision
Add vessel length	69 to 75 min (worse)	rejected
Add time of day and season	about $\pm$0.5 min, no consistent direction	rejected
Replace with Bi-GRU	74.9 vs 68.4 min on the same points (worse)	rejected

7 A berth planning API for Just In Time arrival

A prediction only saves fuel if it changes what a ship does. The model is therefore integrated with a planning API, written in Java (Spring Boot) over a MySQL database, that implements the berth planning side of a terminal operating system (Figure 1). It is the larger part of the codebase and is where the Just In Time logic lives.

7.1 Data model

The database follows one idea common to terminal systems: record the things that happen, attached to the things that exist (Table 4). Its nine tables are created and changed only through versioned Flyway migrations, and the application validates its entities against them at start up. The database is seeded with one demo port, Florø, the busiest mooring area in the AIS data, with four berths at mooring positions observed there; terminal names and berth dimensions are invented.

Table 4. The planning data model.

Table	Kind	One row is
port, terminal, quay, berth	master data	the infrastructure; a berth carries its physical limits (depth, length, beam, draft) and coordinates
vessel	master data	a ship, with optional IMO number, MMSI and dimensions
vessel_visit	transaction	one call of a vessel at the port
berth_allocation	transaction	a berth window given to a visit
eta_prediction	history	an ETA the model produced for a visit, with the inputs it used
visit_event	history	a timestamp reported for a visit, who reported it and when

Three design choices matter for planning. **Occupancy is derived, never stored**: a berth is occupied because it has an open allocation, which can express “occupied from 14:00” rather than only “occupied now”. **Every time has a kind**: estimated, requested, planned or actual, held on the visit and in an append only ledger. **The database enforces the rules itself**: a generated column with a unique index allows only one open allocation per berth and one open visit per vessel, and berth rows are locked while they are being allocated, so concurrent requests cannot double book a berth, even across several running copies of the service.

7.2 Planning a call

Berth or queue. Each position report opens or updates a vessel visit and stores the model's estimate. If a berth is free, the visit is allocated the first free one and its requested arrival equals its predicted arrival. If none is free, the ship is placed in a virtual queue. The first waiting ship is expected to get the berth with the earliest planned end, the second the next, and so on; its requested arrival is the later of that time and its own ETA. A visit's stay is its planned service time when one is given, and otherwise the UNCTAD median time in port for its vessel type [7]: 0.70 days for container ships, 2.05 for dry bulk, 0.94 for tankers and 0.97 otherwise.

Stable requests. A queued ship's requested arrival is re-issued only when it moves by at least 15 minutes, so small ETA updates do not flood the ship with new requests. When a ship departs or cancels, its berth passes to the ship that has waited longest within the same database transaction.

Speed advisory. A queued ship is advised the speed that brings it in at the requested time rather than early:

$$v_{\text{adv}} = \max\left(\frac{d}{\Delta t},\ v_{\text{econ}}\right), \quad \text{issued only if } v_{\text{adv}} < v_{\text{current}}, \tag{2}$$

where Δt is the time until the requested arrival and v_{econ} is a floor taken from published economical speeds (10 knots for container and bulk ships, from the worked example in [2] and carriers' published slow steaming ranges; 8.5 knots for tankers; 5 knots otherwise). This is the Just In Time step: the ship is asked to slow down rather than wait at anchor. The advisory is advice only; the ship stays in control.

7.3 The berth conversation

The exchange between ship and terminal follows the berth planning process in DCSA's Industry Blueprint: the ship estimates, the terminal requests, and the ship accepts, replans or omits the call (Figure 3). A new request from the terminal cancels an earlier acceptance, and an acceptance of a time that has since been replaced is refused. Table 5 lists the endpoints.

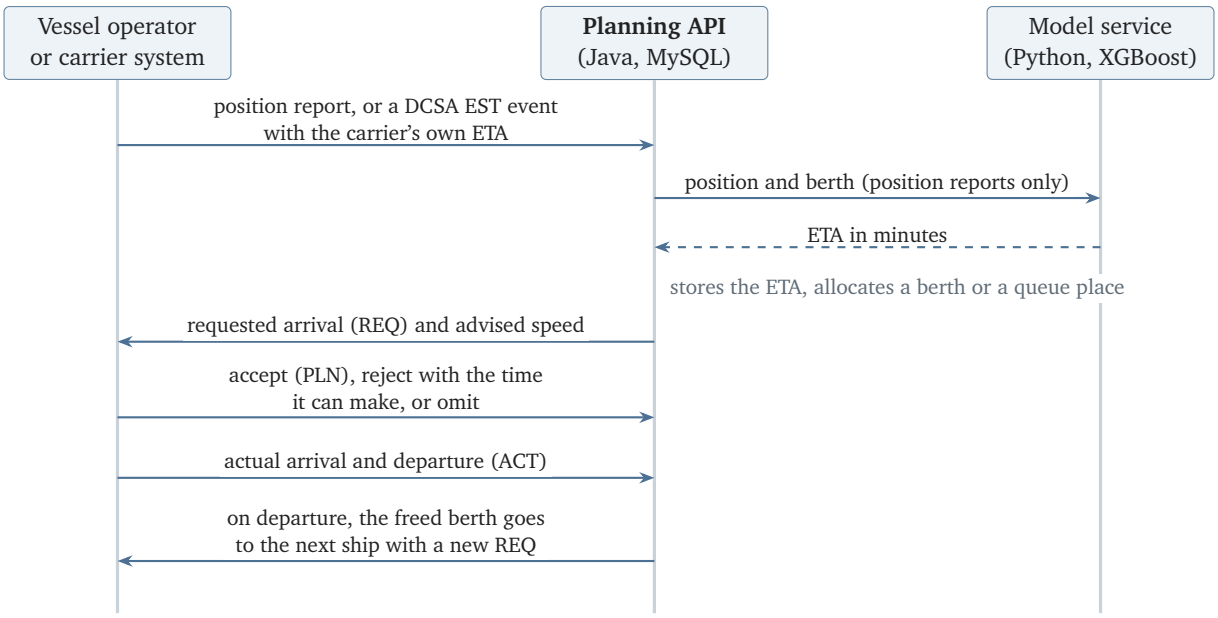


Figure 3. The berth conversation implemented by the API. Codes in brackets are the DCSA time classifiers carried by each message.

Table 5. API endpoints of the planning service.

Endpoint	Purpose
`POST /api/ships/schedule`	position in; model ETA, berth or queue place, requested arrival, advised speed out
`GET /api/ships/berths`	berth occupancy
`PUT /api/ships/berths/{id}/release`	free a berth; it passes to the head of the queue
`POST /api/operator/eta`	ask the model for an ETA without scheduling
`POST /api/operator/vessels/{id}/accept and /reject, /omit`	the operator's reply to a requested arrival
`GET /api/port-call/v2/events`	the timestamp ledger as DCSA Port Call 2.0 events
`POST /api/port-call/v2/events`	receive DCSA berth events from a carrier's system

7.4 Alignment with the DCSA Port Call standard

The ledger is published and received as DCSA Port Call 2.0 events [9]. The implementation follows the standard's field names, nesting and codes (EST, REQ, PLN and ACT; BERTH, ARRI and DEPA), its `API-Version` and `Next-Page-Cursor` headers, cursor paging and error format, and rejects unsupported query filters. Incoming events are applied one at a time, each in its own transaction, and the reply points to any event that could not be applied by its position in the list. Vessels are matched by IMO number, then MMSI, then name. The model's own ETAs are kept out of the published feed, because in the standard an estimated arrival at berth is the carrier's to send.

The alignment is partial, and the project is not certified or tested against DCSA's conformance suite. Not yet covered:

- services other than berthing (pilotage, towage and cargo operations are reported as not processed);
- the standard's separation of a port call from the terminal calls within it, since a visit is one record here;
- facility codes, since the terminal codes are invented;
- authentication and subscriber registration, which the standard leaves to the implementer.

7.5 Testing

The planning service has 54 automated tests: 19 on berth scheduling, 13 on the API controllers, 13 on DCSA event publishing, receiving and endpoints, 5 on the schema migrations and 4 on the planning rules. They run against a throwaway MySQL schema that is rebuilt on every run, so each test starts from a known state.

8 Limitations

- **The label includes idle time.** Arrival is the moment the crew sets the status to moored, which comes well after the ship has stopped: in the last 30 minutes before arrival, 71% of reports are below 5 knots. The model therefore carries a late bias close to port. A label based on speed and position near the berth would be cleaner.
- **Narrow data.** One coastline, one season, mostly coastal vessels and 688 voyages. Generalisation to other ports, larger ships and ocean approaches is untested.
- **Truncated approaches.** Voyages are extracted within each daily file, so an approach that crosses midnight UTC is cut short.

- **Point estimates only.** The model gives no uncertainty, yet the error beyond four hours (about 112 minutes) is large enough that a berth plan should be built on intervals, not single values.
- **Emissions effect not yet measured.** The speed advisory is designed to cut fuel use, but the fuel and CO_2 it would save have not been estimated.
- **Rule based planning.** Allocation is first fit without checking that a ship fits the berth or matches the terminal type, the queue does not reorder when a ship runs late, stay times are published medians rather than measurements of this port, and the endpoints have no authentication.

9 Next steps

1. **Quantify the emissions effect.** Combine the AIS speed profiles with a propulsion power model to estimate the fuel and CO_2 that the advisories would have saved, so the benefit is measured rather than assumed.
2. **Probabilistic ETAs feeding an optimiser.** Replace point estimates with quantile predictions and replace first fit allocation with a berth allocation formulation that minimises waiting under that uncertainty.
3. **Cleaner labels and broader data.** Redefine arrival from motion, process approaches across day boundaries and add ports and seasons.

Throughout, the working practice has been the same: derive labels from data rather than assume them, split so the test reflects real use, test literature recommendations instead of adopting them, and keep negative results in the record. The full pipeline, runbook and experiment code are in the repository.

References

[1] IMO GreenVoyage2050 Project and Global Industry Alliance to Support Low Carbon Shipping (2020). *Just In Time Arrival Guide: Barriers and Potential Solutions*. International Maritime Organization, London.

[2] Global Industry Alliance to Support Low Carbon Shipping, MarineTraffic and Energy and Environmental Research Associates (2022). *Just In Time Arrival: Emissions Reduction Potential in Global Container Shipping*. IMO GreenVoyage2050 Project, London.

[3] Jiang, S., Liu, L., Peng, P., Xu, M. and Yan, R. (2025). Prediction of vessel arrival time to port: a review of current studies. *Maritime Policy & Management*, published online.

[4] Li, H., Jiao, H. and Yang, Z. (2023). AIS data-driven ship trajectory prediction modelling and analysis based on machine learning and deep learning methods. *Transportation Research Part E: Logistics and Transportation Review*, 175, 103152. doi:10.1016/j.tre.2023.103152

[5] Abdi, A. and Amrit, C. (2024). Enhancing vessel arrival time prediction: A fusion-based deep learning approach. *Expert Systems with Applications*, 252, 123988. doi:10.1016/j.eswa.2024.123988

[6] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785 to 794.

[7] UNCTAD (2019). *Review of Maritime Transport 2019*. United Nations Conference on Trade and Development, Geneva.

[8] International Maritime Organization (2023). *Guidelines for Harmonized Communication and Electronic Exchange of Operational Data for Port Calls*. FAL.5/Circ.52.

[9] Digital Container Shipping Association. *Port Call Standard*, version 2.0. dcsa.org/standards/port-call